

Test Plan

S2184546

January 28, 2025

1 Drones must avoid no-fly zones

Lifecycle Approach: Verify

- This requirement involves ensuring that the implemented flight path logic respects no-fly zones by thoroughly testing and validating the system against defined constraints.

Type: Functional and Safety Requirement

Priority: High

Principles Applied:

1. **Partition:** Break down the requirement into smaller subproblems:
 - Identify no-fly zones from input data.
 - Calculate valid flight paths avoiding these zones.
 - Validate the computed path against constraints.
2. **Sensitivity:** Design tests that ensure faults in path calculations consistently manifest in failures:
 - Use boundary cases where paths closely skirt no-fly zones to detect errors reliably.
 - Introduce overlapping or highly complex no-fly zones to stress the algorithm.
3. **Visibility:** Incorporate visual validation: Generate GeoJSON output files for the flight paths and overlay them on a visual map for easy inspection of no-fly zone avoidance.

Testing Tasks:

- **Unit Testing:**
 - Test the pathfinding algorithm with edge cases, ensuring calculated paths never cross no-fly zones.
 - Use synthetic data to model simple and complex no-fly zones.
- **Integration Testing:** Verify the interaction between the pathfinding module and no-fly zone data fetched from the API.
- **System Testing:**
 - Simulate real-world scenarios with multiple orders and varying no-fly zones.
 - Visualise flight paths using geojson.io to confirm adherence to no-fly zone constraints.

Scaffolding and Instrumentation:

- Build a simulator for generating test cases, including randomized and extreme no-fly zone configurations.
- Use visualisation tools for validating paths against constraints.

2 Must Dynamically Fetch Data from a REST API

Lifecycle Approach: Create

- This requirement emphasises building the functionality to fetch, parse, and integrate data from the REST API.

Type: Functional and Integration Requirement

Priority: Medium

Principles Applied:

1. **Redundancy:** Add validation checks on fetched data to ensure consistency and correctness: Verify that no-fly zone data and order details conform to the expected format.
2. **Restriction:** Simplify testing by restricting the test environment: Use a mock API server to control test scenarios, including malformed data or timeout responses.
3. **Feedback:** Incorporate logging for early detection of API-related issues: Capture API responses and errors for debugging and performance improvement.

Testing Tasks:

- **Unit Testing:** Validate parsing of API responses, including both valid and invalid JSON inputs.
- **Integration Testing:** Test the end-to-end interaction with the API under various conditions, such as latency or malformed responses.
- **System Testing:** Execute the entire application, using live or simulated API endpoints, and validate data integration with other modules.

Scaffolding and Instrumentation:

- Use tools like WireMock to create mock APIs simulating realistic and edge-case responses.
- Log all API interactions to assist in debugging and analysis.

3 Must Output Flight Plan in Less Than 60 Seconds

Lifecycle Approach: Monitor

- Since this is a performance requirement, the focus is on tracking and maintaining system performance after deployment.

Type: Performance Requirement

Priority: High

Principles Applied:

1. **Sensitivity:** Profile bottlenecks in the algorithm to ensure small inefficiencies are easily observable: Use timing logs to monitor execution times for each function.
2. **Partition:** Test individual modules (e.g., pathfinding, file generation) for performance before integrating them: Optimise each module to avoid cascading delays.
3. **Feedback:** Use timing data from system tests to prioritize optimisation efforts: Continuously refine bottlenecks identified in earlier runs.

Testing Tasks:

- **Unit Testing:** Measure performance of individual components (e.g., pathfinding, data parsing).
- **Performance Testing:**
 - Test overall execution time with varying number of orders.
 - Introduce complex scenarios (e.g., dense no-fly zones) to evaluate worst-case performance.
 - Ensure that the system will timeout if a path cannot be calculated in 60 seconds.
- **System Testing:** Simulate end-to-end operations under normal and high-load conditions.

Scaffolding and Instrumentation:

- Add instrumentation for detailed timing logs at the module and system levels.
- Use performance profiler tools.